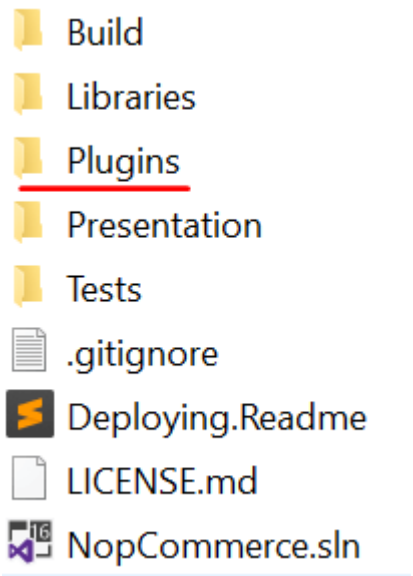
How to write a widget for nopCommerce

10-13 minutes

Para ampliar la funcionalidad de nopCommerce, se utilizan widgets. Existen varios tipos de widgets, como Swiper y Google Analytics , que ya están incluidos en el repositorio de nopCommerce. El marketplace de nopCommerce ya cuenta con diversos widgets (tanto gratuitos como de pago) que pueden satisfacer tus necesidades. Si no has encontrado el que buscas, estás en el lugar correcto, ya que este artículo te guiará en el proceso de creación de un widget a tu medida.

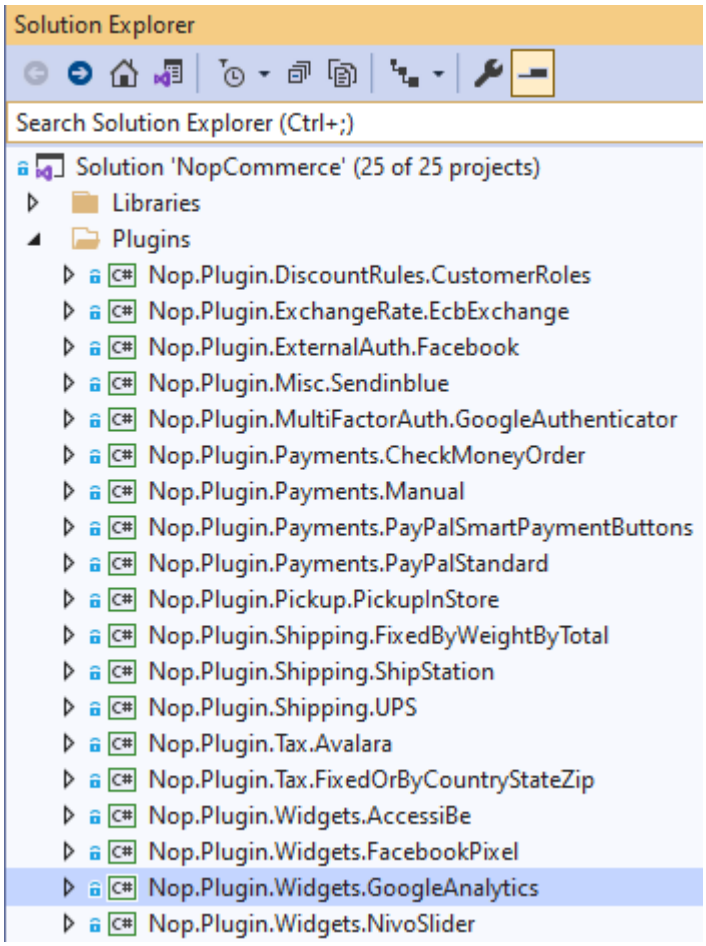
1. Para empezar, crea un nuevo proyecto **de biblioteca de clases** en la solución. Se recomienda ubicar tu widget en el directorio **Plugins** , situado en la carpeta raíz del código fuente, donde ya se encuentran otros widgets y plugins.



Nota

No confunda este directorio con el que se encuentra en *Presentation\Nop.Web* . El directorio *Plugins* , dentro de *Nop.Web* , contiene los archivos compilados del plugin.

Un nombre recomendado para un proyecto de widget es `Nop.Plugin.Widgets.{Name}`. `{Name}`es el nombre de su widget (por ejemplo, " **GoogleAnalytics** "). Por ejemplo, *el widget de Google Analytics* tiene el siguiente nombre: `Nop.Plugin.Widgets.GoogleAnalytics`. Pero tenga en cuenta que no es un requisito. Y puede elegir cualquier nombre para un widget. Por ejemplo, " *MyFirstNopWidget* ". La estructura del directorio *Plugins* de una solución se ve así.



2. Una vez creado el proyecto del widget, se debe actualizar el contenido del archivo **.csproj** utilizando cualquier editor de texto disponible. Reemplace el contenido con el siguiente:

```
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <TargetFramework>net9.0</TargetFramework>
        <Copyright>SOME_COPYRIGHT</Copyright>
        <Company>YOUR_COMPANY</Company>
        <Authors>SOME_AUTHORS</Authors>

<PackageLicenseUrl>PACKAGE_LICENSE_URL</PackageLicenseUrl>

<PackageProjectUrl>PACKAGE_PROJECT_URL</PackageProjectUrl>
        <RepositoryUrl>REPOSITORY_URL</RepositoryUrl>
        <RepositoryType>Git</RepositoryType>

<OutputPath>$(SolutionDir)\Presentation\Nop.Web\Plugins\WIDGET_OUTP
        <OutDir>$(OutputPath)</OutDir>
        <!--Set this parameter to true to get the dlls copied
from the NuGet cache to the output of your    project. You need
to set this parameter to true if your plugin has a nuget package
to ensure that   the dlls copied from the NuGet cache to the
output of your project-->
```

```
<CopyLocalLockFileAssemblies>true</CopyLocalLockFileAssemblies>
        <ImplicitUsings>enable</ImplicitUsings>
    </PropertyGroup>
    <ItemGroup>
        <ProjectReference
Include="$(SolutionDir)\Presentation\Nop.Web.Framework\Nop.Web.Fram
/>
        <ClearPluginAssemblies
Include="$(SolutionDir)\Build\ClearPluginAssemblies.csproj" />
    </ItemGroup>
    <!-- This target execute after "Build" target -->
    <Target Name="NopTarget" AfterTargets="Build">
        <!-- Delete unnecessary libraries from plugins path -->
        <MSBuild Projects="@(ClearPluginAssemblies)"
Properties="PluginPath=$(OutDir)" Targets="NopClear" />
    </Target>
</Project>
```

Nota

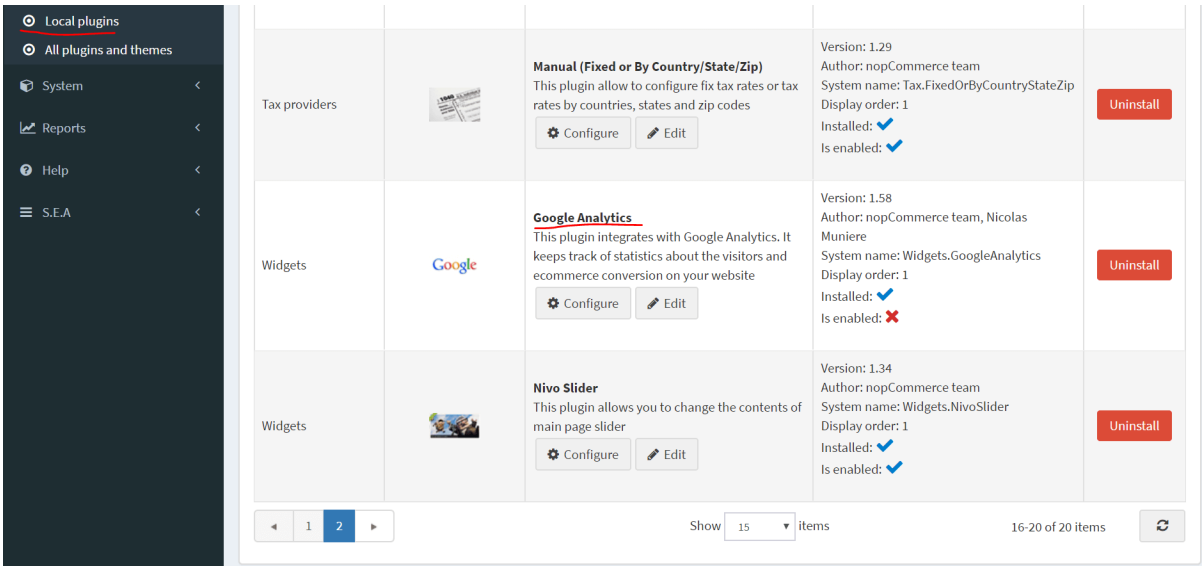
El **valor de WIDGET_OUTPUT_DIRECTORY** debe sustituirse por el nombre del plugin, por ejemplo *Widgets.GoogleAnalytics* .

3. Tras actualizar el archivo *.csproj , se debe añadir el archivo* **plugin.json** , necesario para un widget. Este archivo contiene metainformación que describe el widget. Simplemente copie este archivo de cualquier otro plugin o widget existente y modifíquelo según sus necesidades. Para obtener más información sobre el `plugin.json`archivo, consulte la documentación de plugin.json .

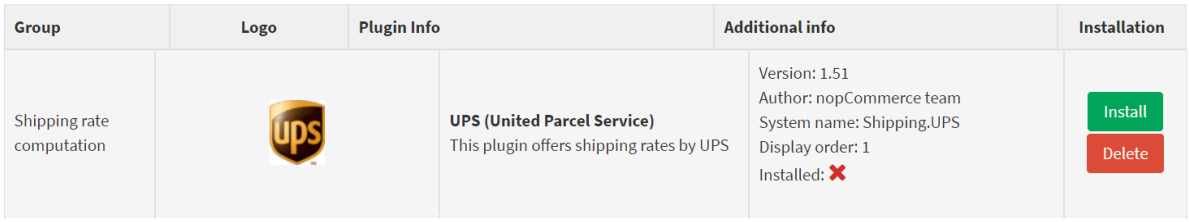
   El último paso necesario es crear una clase que implemente **BasePlugin** (espacio de nombres *Nop.Core.Plugins*) y la interfaz **IWidgetPlugin** (espacio de nombres *Nop.Services.Cms*). IWidgetPlugin permite crear widgets. Estos widgets se muestran en ciertas partes del sitio web. Por ejemplo, puede ser un bloque de chat en vivo en la esquina inferior derecha.

Gestión de solicitudes. Controladores, modelos y vistas.

Ahora puedes ver el widget accediendo a **Área de administración** → **Configuración** → **Complementos locales** .



Cuando se instala un plugin/widget, verá el botón **Desinstalar** . *Para mejorar el rendimiento, es recomendable desinstalar los plugins/widgets que no sean necesarios* .



Aparecerán los botones **Instalar** y **Eliminar** cuando un plugin/widget no esté instalado o desinstalado. *Al eliminarlo, se borrarán los archivos físicos del servidor* .

Pero, como ya habrás adivinado, nuestro widget no hace nada. Ni siquiera tiene una interfaz de usuario para su configuración. Vamos a crear una página para configurar el widget.

Lo que debemos hacer ahora es crear un controlador, un modelo, una vista y un componente de vista.

- **Los controladores MVC** son responsables de responder a las solicitudes realizadas a un sitio web *ASP.NET MVC* . Cada solicitud del navegador se asigna a un controlador específico.
- Una vista contiene el marcado **HTML** y el contenido que se envía al navegador. Una vista es el equivalente a una página cuando se trabaja con una aplicación *ASP.NET MVC* .
- Un componente de vista que implementa **NopViewComponent** y que contiene la lógica y el código para renderizar una vista.
- El **modelo MVC** contiene toda la lógica de la aplicación que no está contenida en una vista o un controlador.

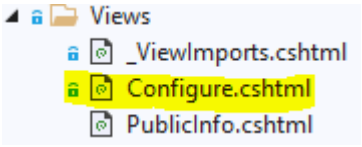
Así que comencemos:

1. Crea el modelo. Agrega una `Models`carpeta en el nuevo widget y luego agrega una nueva clase de modelo que se ajuste a tus necesidades.

2. Crea la vista. Agrega una `Views`carpeta en el nuevo widget y luego agrega un `cshtml`archivo llamado `Configure.cshtml`. Establece la propiedad " **Build Action** " del archivo de vista en " **Content** " y la propiedad " **Copiar al directorio de salida** " en " **Copiar siempre** ". Ten en cuenta que la página de configuración debe usar el diseño " **_ConfigurePlugin ".**

```
@{
    Layout = "_ConfigurePlugin";
}
```

3. Asegúrate también de tener el archivo **_ViewImports.cshtml** en tu `Views`directorio. Puedes copiarlo desde cualquier otro plugin o widget existente.



4. Crea el controlador. Agrega una `Controllers`carpeta en el nuevo widget y luego agrega una nueva clase de controlador. Una buena práctica es nombrar los controladores de plugins `Widgets{Name}Controller.cs`. Por ejemplo, **WidgetsGoogleAnalyticsController** . Por supuesto, no es un requisito nombrar los controladores de esta manera, sino solo una recomendación. Luego crea un método de acción apropiado para la página de configuración (en el área de administración). Llamémoslo `Configure`. Prepara una clase de modelo y pásala a la siguiente vista usando una ruta de vista física: `~/Plugins/{PluginOutputDirectory}/Views/Configure.cshtml`.

```
public async Task<IActionResult> Configure()
{
    if (!await
_permissionService.AuthorizeAsync(StandardPermissionProvider.Manage
        return AccessDeniedView();

    //load settings for a chosen store scope
    var storeScope = await
_storeContext.GetActiveStoreScopeConfigurationAsync();
    var myWidgetSettings = await
_settingService.LoadSettingAsync<MyWidgetSettings>(storeScope);

    var model = new ConfigurationModel
    {
        // configuration model settings here
    };

    if (storeScope > 0)
    {
        // override settings here based on store scope
```

```
    }

    return
View("~/Plugins/Widgets.MyFirstNopWidget/Views/Configure.cshtml",
model);
}
```

5. Utilice los siguientes atributos para su método de acción:

```
[AutoValidateAntiforgeryToken]
[AuthorizeAdmin] //confirms access to the admin panel
[Area(AreaNames.Admin)] //specifies the area containing a
controller or action
[AdminAntiForgery] //Helps prevent malicious scripts from
submitting forged page requests.
```

Por ejemplo, abre el `GoogleAnalytics`widget y observa su implementación `WidgetsGoogleAnalyticsController`. Luego, para cada widget que tenga una página de configuración, debes especificar una URL de configuración. La clase base llamada **BasePlugin** tiene `GetConfigurationPageUrl`un método que devuelve una URL de configuración:

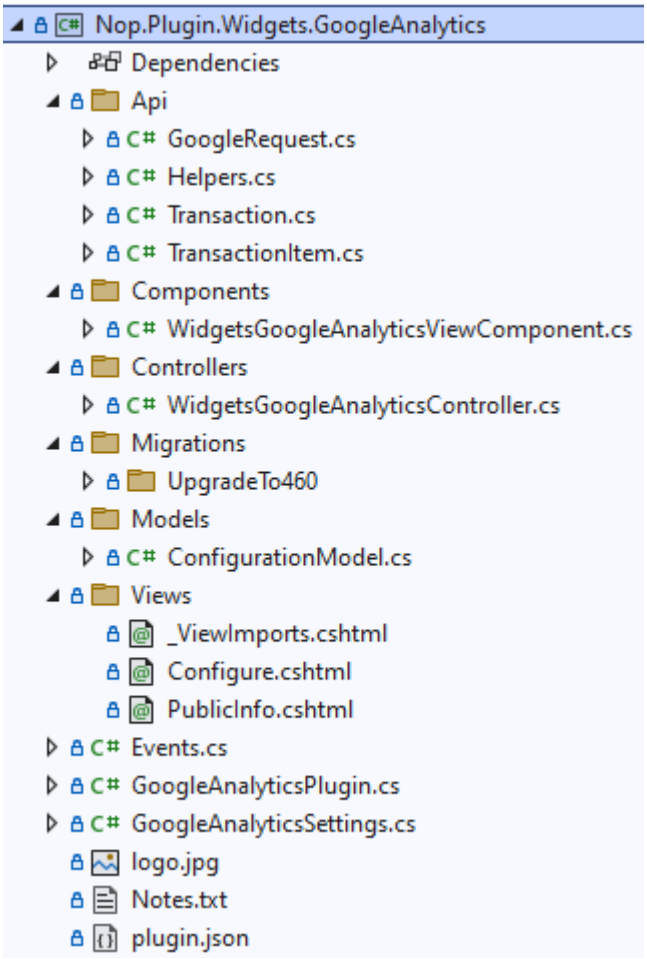
```
public override string GetConfigurationPageUrl()
{
    return $"
{_webHelper.GetStoreLocation()}Admin/{CONTROLLER_NAME}/{ACTION_NAME
}
```

Donde `{CONTROLLER_NAME}`es el nombre de tu controlador y `{ACTION_NAME}`es el nombre de la acción (normalmente es "Configurar"). Cada widget debe especificar una lista de zonas de widgets. Una clase base llamada **IWidgetPlugin** tiene el `GetWidgetZones`método que devuelve una lista de zonas de widgets donde se renderizará.

```
public Task<IList<string>> GetWidgetZonesAsync()
{
    return Task.FromResult<IList<string>>(new List<string>
{PublicWidgetZones.HeadHtmlTag });
}
```

Puedes encontrar una lista de zonas de widgets públicas en este enlace y zonas de widgets de administración en este otro . Además `GetWidgetZonesAsync`, **IWidgetPlugin** tiene `GetWidgetViewComponentName`un método que devuelve el nombre del ViewComponent. Acepta el nombre de " *widgetZone* " como parámetro y se puede usar para renderizar diferentes vistas según la zona de widgets seleccionada.

```
public string GetWidgetViewComponentName(string widgetZone)
{
    return "MyFirstWidget";
}
```



Manejo de los métodos "InstallAsync" y "UninstallAsync".

Este paso es opcional. Algunos widgets pueden requerir lógica adicional durante su instalación. Por ejemplo, un widget puede insertar nuevos recursos de localización o valores de configuración. Por lo tanto, abra su implementación **de IWidgetPlugin** (en la mayoría de los casos derivará de la clase **BasePlugin**) y sobrescriba los siguientes métodos:

1. **InstallAsync** . Este método se invocará durante la instalación del complemento. Aquí puede inicializar cualquier configuración, insertar nuevos recursos de localización o crear nuevas tablas de base de datos (si es necesario).

```
public override async Task InstallAsync()
{
    // custom logic like adding settings, locale resources, and
database table(s) here
```

```
    await base.InstallAsync();
}
```

2. **UninstallAsync** . Este método se invocará durante la desinstalación del complemento. Puede eliminar configuraciones, recursos de localización o tablas de base de datos previamente inicializados mediante el widget durante la instalación.

```
public override async Task UninstallAsync()
{
    // custom logic like removing settings, locale resources, and
database table(s) which was created during widget installation

    await base.UninstallAsync();
}
```

Importante

Si sobrescribe alguno de estos métodos, no oculte su implementación base: **base.InstallAsync()** y **base.UninstallAsync()** , que se han marcado en las imágenes anteriores.